



COMP30026

Models of Computation

Cezary Kaliszyk and William Umboh

Lecture 14

Regular Expressions



Reminder: Regular Expressions

The **regular expressions** over an alphabet $\Sigma = \{a_1, \dots, a_n\}$ are given by the grammar

$$\begin{array}{lcl} \text{regexp} & \rightarrow & a_1 \quad | \quad \dots \quad | \quad a_n \quad | \quad \epsilon \quad | \quad \emptyset \\ & & | \quad \text{regexp} \cup \text{regexp} \quad | \quad \text{regexp} \circ \text{regexp} \quad | \quad \text{regexp}^* \end{array}$$



Reminder: Regular Expressions

The **regular expressions** over an alphabet $\Sigma = \{a_1, \dots, a_n\}$ are given by the grammar

$$\begin{array}{lcl} \text{regexp} & \rightarrow & a_1 \mid \dots \mid a_n \mid \epsilon \mid \emptyset \\ & & \mid \text{regexp} \cup \text{regexp} \mid \text{regexp} \circ \text{regexp} \mid \text{regexp}^* \end{array}$$

Semantics:

$$\begin{array}{ll} L(a) & = \{a\} \\ L(\epsilon) & = \{\epsilon\} \\ L(\emptyset) & = \emptyset \\ L(R_1 \cup R_2) & = L(R_1) \cup L(R_2) \\ L(R_1 R_2) & = L(R_1) \circ L(R_2) \\ L(R^*) & = L(R)^* \end{array}$$



Regular Expressions to NFAs

Theorem

A language is regular iff it can be described by a regular expression.

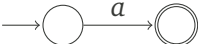
Regular Expressions to NFAs

Theorem

A language is regular iff it can be described by a regular expression.

Proof idea ($\Leftarrow$): Construct NFA from regular expression R .
Bottom up. (Formally: induction).

Base cases: $R = a \in \Sigma$, $R = \epsilon$, or $R = \emptyset$.

If $R = a$: Construct 

If $R = \epsilon$: Construct 

If $R = \emptyset$: Construct 

Inductive step: $R = R_1 \cup R_2$, $R = R_1 \circ R_2$, or $R = R_1^*$. Use the constructions for closure under regular operations.

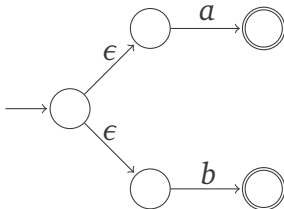
Regular Expressions to NFAs: Example

Convert $(a \cup b)^*bc$ to an NFA.

Start from innermost expressions and work out:

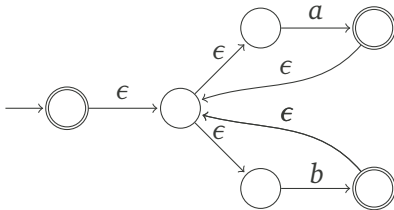


So an NFA for $a \cup b$ is:

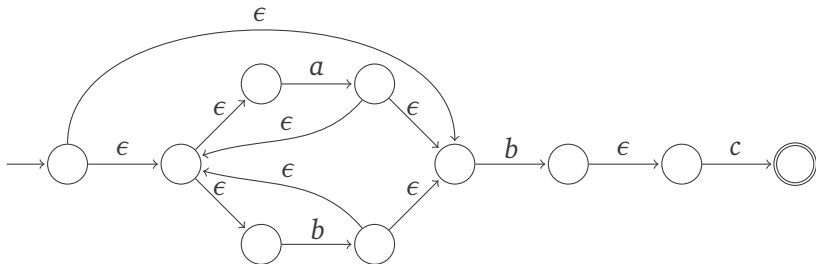


NFAs from Regular Expressions

Use star construction to get NFA for $(a \cup b)^*$:



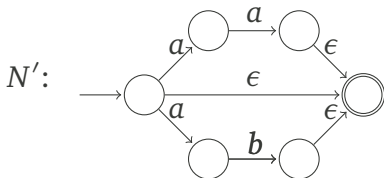
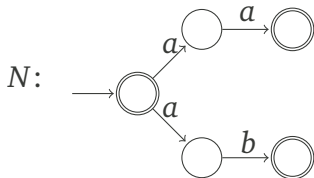
Finally, for $(a \cup b)^*bc$, we get:



Regular Expressions from NFAs

Other way ($\implies$).

Given an NFA N , we can build an equivalent NFA with at most **one accept state**.

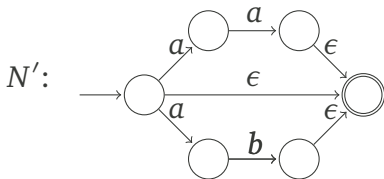
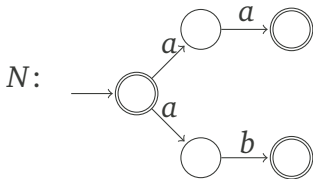


Regular Expressions from NFAs

Other way ($\implies$).

Given an NFA N , we can build an equivalent NFA with at most **one accept state**. We transform $N = (Q, \Sigma, \delta, I, F)$ to $N' = (Q \cup \{q_f\}, \Sigma, \delta', I, \{q_f\})$ like so: If $|F| \leq 1$, do nothing. Otherwise add a **new** q_f and ϵ transitions to q_f from each state in F . q_f becomes the only accept state:

$$\delta'(q, v) = \begin{cases} \delta(q, v) \cup \{q_f\} & \text{if } q \in F \text{ and } v = \epsilon \\ \delta(q, v) & \text{otherwise} \end{cases}$$





Regular Expressions from NFAs

Start by making sure the NFA has a single accept state.

Repeatedly eliminate states that are neither start nor accept states.

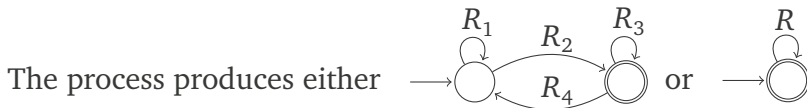
In the process, arcs get labelled with regular expressions.

Regular Expressions from NFAs

Start by making sure the NFA has a single accept state.

Repeatedly eliminate states that are neither start nor accept states.

In the process, arcs get labelled with regular expressions.



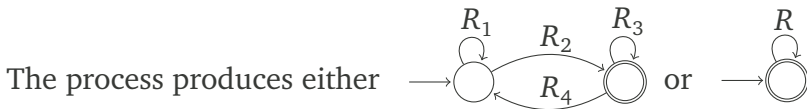
Note that some R s may be ϵ or $\emptyset$.

Regular Expressions from NFAs

Start by making sure the NFA has a single accept state.

Repeatedly eliminate states that are neither start nor accept states.

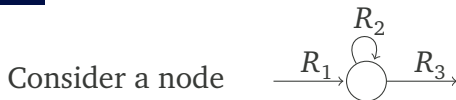
In the process, arcs get labelled with regular expressions.



Note that some R s may be ϵ or $\emptyset$.

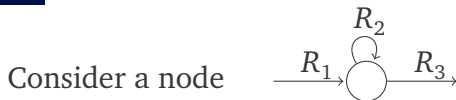
We get $(R_1 \cup R_2 R_3^* R_4)^* R_2 R_3^*$ in the first case; R^* in the second.

The State Elimination Process



Any such pair of incoming/outgoing arcs get replaced by a single arc that **bypasses** the node. The new arc gets the label $R_1R_2^*R_3$.

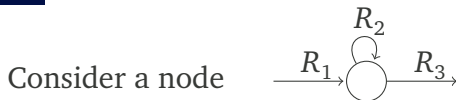
The State Elimination Process



Any such pair of incoming/outgoing arcs get replaced by a single arc that **bypasses** the node. The new arc gets the label $R_1R_2^*R_3$.

If there are m incoming and n outgoing arcs, these arcs are replaced by $m \times n$ bypassing arcs when the node is removed.

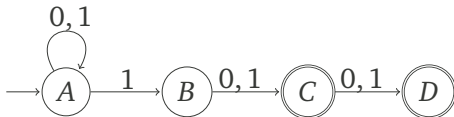
The State Elimination Process



Any such pair of incoming/outgoing arcs get replaced by a single arc that **bypasses** the node. The new arc gets the label $R_1R_2^*R_3$.

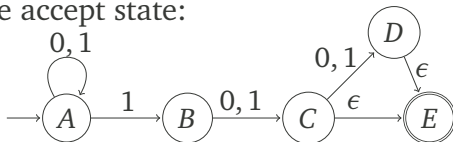
If there are m incoming and n outgoing arcs, these arcs are replaced by $m \times n$ bypassing arcs when the node is removed.

Let us illustrate the process on this example:



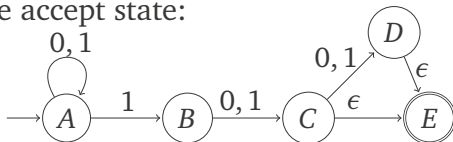
State Elimination Example

Create a single accept state:

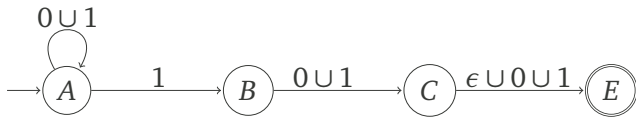


State Elimination Example

Create a single accept state:

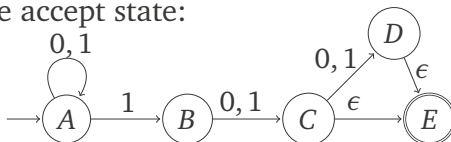


Eliminate D (and use regular expressions with all arcs):

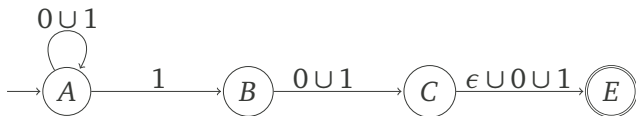


State Elimination Example

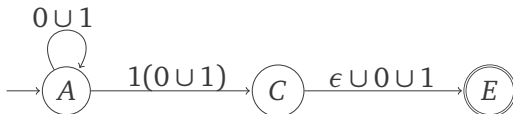
Create a single accept state:



Eliminate D (and use regular expressions with all arcs):

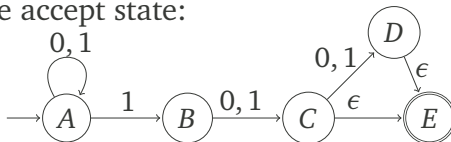


Now eliminate B :

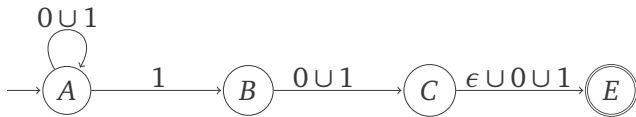


State Elimination Example

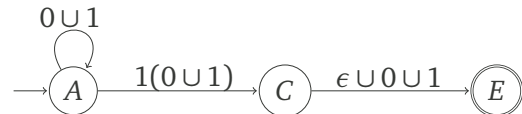
Create a single accept state:



Eliminate D (and use regular expressions with all arcs):



Now eliminate B :



```

graph LR
    start(( )) --> A((A))
    A -- "0 ∪ 1" --> A
    A -- "1(0 ∪ 1)" --> C((C))
    C -- "ε ∪ 0 ∪ 1" --> E(((E)))
    style start fill:none,stroke:none
  
```

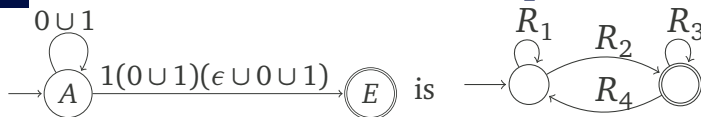
and then C :



```

graph LR
    start(( )) --> A((A))
    A -- "0 ∪ 1" --> A
    A -- "1(0 ∪ 1)(ε ∪ 0 ∪ 1)" --> E(((E)))
    style start fill:none,stroke:none
  
```

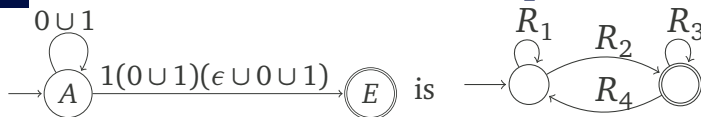
State Elimination Example



with

- $R_1 = 0 \cup 1$
- $R_2 = 1(0 \cup 1)(\epsilon \cup 0 \cup 1)$
- $R_3 = R_4 = \emptyset$

State Elimination Example



with

- $R_1 = 0 \cup 1$
- $R_2 = 1(0 \cup 1)(\epsilon \cup 0 \cup 1)$
- $R_3 = R_4 = \emptyset$

Hence the instance of the general “recipe”
 $(R_1 \cup R_2 R_3^* R_4)^* R_2 R_3^*$ is

$$(0 \cup 1)^* 1(0 \cup 1)(\epsilon \cup 0 \cup 1)$$

More details: Sipser (“Modules/Resources” on Canvas)



Some Useful Laws for Regular Expressions

$$A \cup A = A$$

$$A \cup B = B \cup A$$

$$(A \cup B) \cup C = A \cup (B \cup C) = A \cup B \cup C$$

$$(A \circ B) \circ C = A \circ (B \circ C) = A \circ B \circ C$$

$$\emptyset \cup A = A \cup \emptyset = A$$

$$\epsilon \circ A = A \circ \epsilon = A$$

$$\emptyset \circ A = A \circ \emptyset = \emptyset$$

More Useful Laws for Regular Expressions

$$(A \cup B) \circ C = (A \circ C) \cup (B \circ C)$$

$$A \circ (B \cup C) = (A \circ B) \cup (A \circ C)$$

$$(A^*)^* = A^*$$

$$\emptyset^* = \epsilon^* = \epsilon$$

$$(\epsilon \cup A)^* = A^*$$

$$(A \cup B)^* = (A^* B^*)^*$$



Limitations of Finite Automata

Cannot look ahead.

Fixed number of bits of memory.

How many bits to recognise this, without lookahead?

$$\{0^n 1^n \mid n \geq 0\} = \{\epsilon, 01, 0011, 000111, \dots\}$$



Limitations of Finite Automata

Cannot look ahead.

Fixed number of bits of memory.

How many bits to recognise this, without lookahead?

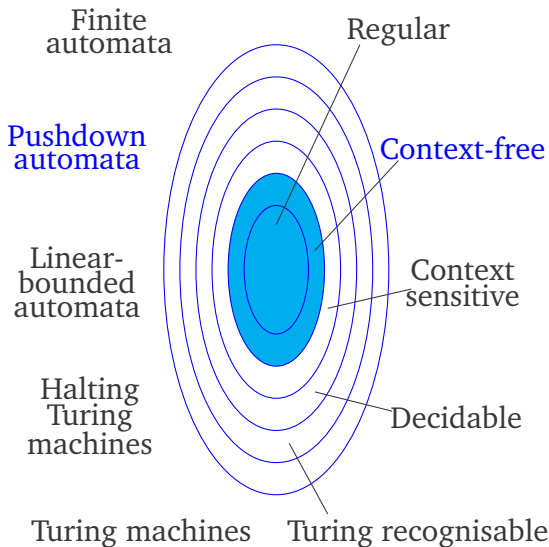
$$\{0^n 1^n \mid n \geq 0\} = \{\epsilon, 01, 0011, 000111, \dots\}$$

Bonus: Is the language $L_1 = \{0^n 1^n \mid 0 \leq n \leq 999999999\}$ regular?

What about

$$L_2 = \left\{ w \mid \begin{array}{l} w \text{ has an equal number of occurrences} \\ \text{of the substrings } 01 \text{ and } 10 \end{array} \right\} ?$$

Machines vs Languages





Context-Free Grammars (CFGs)

Already used for wffs and regular expressions!

Main application: parsing programming languages.

Described as a set of **substitution rules**, or **productions**.

Example:

$$R \rightarrow 0$$

$$R \rightarrow 1$$

$$R \rightarrow \mathbf{eps}$$

$$R \rightarrow \mathbf{empty}$$

$$R \rightarrow R \cup R$$

$$R \rightarrow R \circ R$$

$$R \rightarrow R^*$$



Context-Free Grammars (CFGs)

Already used for wffs and regular expressions!

Main application: parsing programming languages.

Described as a set of **substitution rules**, or **productions**.

Example:

$$R \rightarrow 0$$

$$R \rightarrow 1$$

$$R \rightarrow \mathbf{eps}$$

$$R \rightarrow \mathbf{empty}$$

$$R \rightarrow R \cup R$$

$$R \rightarrow R \circ R$$

$$R \rightarrow R^*$$

Shorthand: $R \rightarrow 0 \mid 1 \mid \mathbf{eps} \mid \mathbf{empty} \mid R \cup R \mid R R \mid R^*$



Generating with Grammars

Consider the grammar G :

$$A \rightarrow 0A0$$

$$A \rightarrow 1A1$$

$$A \rightarrow \epsilon$$

A is a **variable**. 0 and 1 are **terminals**.



Generating with Grammars

Consider the grammar G :

$$A \rightarrow 0A0$$

$$A \rightarrow 1A1$$

$$A \rightarrow \epsilon$$

A is a **variable**. 0 and 1 are **terminals**.

Generation process:

1. Start with LHS of first rule.
2. Replace **one** instance of a variable with RHS of a matching rule.
3. Repeat step 2 until no variables remain.



Derivation Example

Given the grammar G :

$$A \rightarrow 0A0$$

$$A \rightarrow 1A1$$

$$A \rightarrow \epsilon$$

One possible **derivation** is

$$A \Rightarrow 0A0$$

$$\Rightarrow 00A00$$

$$\Rightarrow 001A100$$

$$\Rightarrow 0010A0100$$

$$\Rightarrow 00100100$$

We say G **generates** 00100100.

The intermediate strings are called **sentential forms**.



Context-Free Languages

Definition

The language of a grammar is the set of strings it generates.

Definition

A language is **context-free** iff it is generated by some CFG.

Exercise: write a CFG for the non-regular language $\{0^n 1^n \mid n \geq 1\}$.

Context-Free Languages

Definition

The language of a grammar is the set of strings it generates.

Definition

A language is **context-free** iff it is generated by some CFG.

Exercise: write a CFG for the non-regular language $\{0^n 1^n \mid n \geq 1\}$.

$$A \rightarrow 0A1 \mid 01$$